5.1 Project Overview

Using the **select()** system call, this project will create a **concurrent TCP chat server** in C that is designed in an Event-Driven Programming (EDP) fashion.

This system is based on the client-server model. The server manages multiple client connections at the same time. Each client process runs independently, communicates with the server over TCP Sockets and uses a newline-delimited text protocol.

The server has the primary responsibility for parsing commands, maintaining client states and routing messages as they need to be sent. The features of the system are:

- user authentication with unique usernames
- Create named channels dynamically
- Message broadcasting based on channel
- Private messaging between users
- Users in the channel can be displayed
- Gracefully handle expected and unexpected disconnects

The goals of the project are to demonstrate methods of doing **network programming, concurrency, and protocol creation using low-level system calls.**

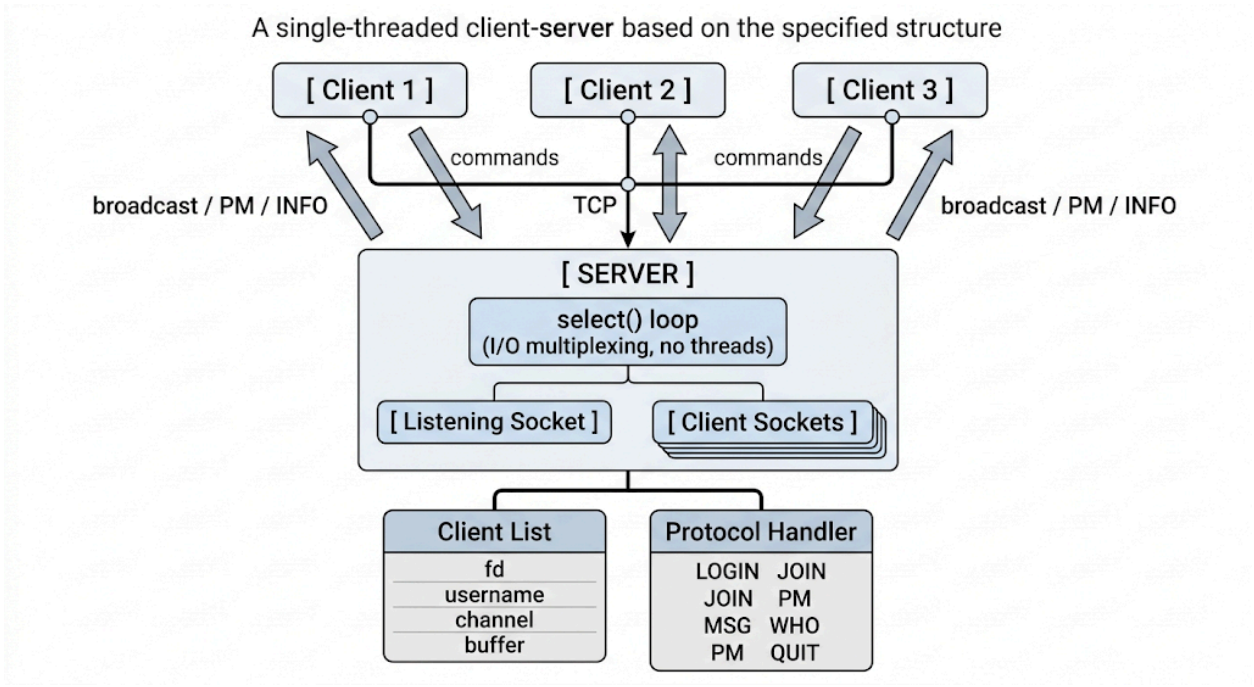
5.2 Build Instructions

This project uses the accompanying Makefile to build.
To compile:
make clean
make
This will create two executables, the **server** and the **client**.
To start the server:
./server <port number>
For example:
./server 5555
To start a client:
./client <hostname> <port number>
For example:
./client 127.0.0.1 5555
How to use this system after you have established a connection:

- Enter your name
- Join the channel by using the following command: /join <channel name>

- Type as you would normally to send a message
- Use the following command to display all users currently in the channel: /who
- When you want to send a private message to another user type the following command: /pm <username> <private message>
- You can disconnect from the server by typing /quit

5.3 Architecture Diagram



5.4 Communication Protocol

All communication between clients and the server follows a **newline-delimited text protocol**, where each message type has a defined encoding, semantics, and error handling behavior.

LOGIN — Client → Server
Encoding: LOGIN <username>\n
Semantics: Authenticates a user by assigning a unique username.
Behavior: Server validates format (non-empty, no spaces/tabs) and uniqueness, then stores username.
Response: OK or ERROR
Errors: Invalid or duplicate usernames → ERROR

JOIN — Client → Server
Encoding: JOIN <channel>\n

Semantics: Moves a user into a channel (created dynamically).
Behavior: Server updates channel, notifies previous channel (if any), and broadcasts join message.
Response: OK + INFO joined channel
Errors: Invalid channel → ERROR

MSG — Client → Server → Channel
Encoding: MSG <text>\n
Semantics: Broadcasts a message to all users in the same channel.
Behavior: Server forwards message only to clients in the same channel.
Output: [channel] username: text
Errors: If not in a channel → INFO Join a channel first

PM — Client → Server → Client
Encoding: PM <username> <text>\n
Semantics: Sends a private message to another user regardless of associated channel.
Behavior: Server locates target and delivers message.
Output: [PM from sender] text / [PM to recipient] text
Errors: Invalid user or format → ERROR

WHO — Client → Server
Encoding: WHO\n
Semantics: Lists users in the current channel.
Behavior: Server returns usernames in the same channel.
Output: USERS <channel>: user1, user2, …
Errors: If not in channel → INFO Join a channel first

QUIT — Client → Server
Encoding: QUIT\n
Semantics: Disconnects the client.
Behavior: Server removes client and notifies channel.
Output: INFO user left channel

INFO — Server → Clients
Encoding: INFO <message>\n
Semantics: System notifications (join/leave/events).

5.5 Concurrency Model

Concurrency Model

The concurrency model for the project is based on a Category 2 model; it uses the select() system call to implement event-driven concurrency. A single-process server architecture without the need for fork() or threads has been used, and all coordination between clients is accomplished using socket-based message passing.

Event Loop

An event loop executes continuously. Each time through the event loop, the event loop builds an fd_set (i.e., a set of file descriptors) based on the listening socket and all currently-connected client sockets. After building the fd_set, the event loop calls:

select(max_fd +1, &read_fds, NULL, NULL, NULL)

If a new connection has been established on the listening socket, the event loop will accept the new connection. If any client sockets have data in their receive buffers ready to be read and passed to the protocol layer for processing through the event loop, they will do so.

Concurrency Behavior

Concurrency is achieved through the use of the select() system call, which allows monitoring of all sockets at once. The server only calls recv() on sockets marked ready by select(), allowing multiple clients to be serviced concurrently by the server with no blocking due to waiting on only one client.

Limitation

The server uses blocking send() calls; therefore, delivery of messages may be delayed due to a slow client connection, but typically will have little impact on performance when using small amounts of data in messages.

Disconnect Handling

The server supports both graceful disconnect operations **QUIT** and abnormally disconnected clients (recv() returns 0 or with an error). In either case, the client is removed from the current channel, the socket is closed, and others in the same channel are notified of this event.

5.6 Error Handling and Robustness

The server is designed to handle common failures and abnormal client behavior without crashing or leaving the system in a suboptimal state. Several potential runtime issues were encountered and handled.

Duplicate username during login

There is a potential issue where a client tries to log into the same username as another already logged in user, which would create confusion when sending messages to the user and have a valid identification of the user. Therefore, upon receiving a login attempt, the server will check ongoing connections to verify if a client is trying to log into a unique username. If there is already a connection with that username, the system will reject the login attempt with an ERROR message, thus, each client logged into the server will have a unique identification.

Invalid client state (e.g., messaging before joining a channel)

Clients can send messages (e.g., via the MSG command) before their state is valid (e.g., before joining a channel). The server verifies the state of the client before processing the command. When a client that is not in any channel attempts to send a message, the server will not process the message and will instead respond with an information message telling the user they must join a channel first. This keeps message routing from being undefined and ensures it is always consistent.

Unexpected client disconnect

An unexpected client disconnect occurs when a client unexpectedly disconnects from the server (e.g., by closing the terminal) without sending the QUIT command. The server will detect the disconnect when the return from a call to recv() is 0. When this happens, the server will consider this to be an unexpected disconnection and will close the associated socket, remove the client from its internal data structures, and notify the other clients in the same channel of the disconnection. The effect of this is to ensure that there are no stale clients remaining in the system and that the state of each channel remains accurate.

Malformed or invalid command

Clients may send commands that do not follow the expected protocol format, such as missing arguments or unknown commands. The protocol parser validates each command before acting on it. If a command is malformed, the server responds with **Error** and continues running without disconnecting the client. This makes the system more robust while still allowing the client to recover and send valid commands.

Input buffer overflow protection

Another potential issue is a client sending a large amount of data without completing a newline-terminated command, which could overflow the input buffer. To handle this, the server checks whether incoming data would exceed the buffer size. If it does, the server sends an **Error** response and disconnects the client. This prevents memory corruption and ensures safe message handling.

5.7 Team Contributions

Iain Campbell

I was responsible for the server-side of the implementation, including server.c, server_utils.c, and server_utils.h. In the server-side implementation, I implemented TCP through socket functions (create a socket, create a TCP port, listen for connections, and accept connections), and implement a select() based event model to provide a way to listen for new connections while simultaneously processing client input. Additionally, I wrote code to create new client connections, track the state of the client (connected or disconnected), handle the broadcasting of messages to all connected clients, and perform low-level I/O operations on a per-socket basis

Soumil Nag

I was responsible for the client and protocol layers, including client.c, protocol_server.c, and protocol_server.h. I implemented the client interface for handling user input and server-generated messages by using a select() based event model. Additionally, I designed and implemented the protocol for parsing commands (LOGIN, JOIN, MSG, PM, WHO, and QUIT) as well as for validating and generating responses to all commands.